

Multilevel Feedback Queue (MLFQ)

MLFQ gives new processes the highest priority. CPU usage can move them to lower priorities. Periodic boosts give all active processes fresh budgets in the top queue.

1. Queues and CPU budgets

The visualizer uses one CPU, whole ticks, and three ready queues. Q0 is highest priority, followed by Q1 and Q2. Processes need only CPU service, with no I/O blocking or cost for switching processes.

<i>Quantity</i>	<i>Meaning</i>
Remaining service	CPU ticks still needed to finish.
Quantum (time slice)	CPU ticks allowed in one turn.
Allotment	Total CPU ticks allowed at a priority before demotion, or renewal in Q2.

Quantum and allotment are independent settings. Their defaults happen to be equal:

<i>Queue</i>	<i>Quantum</i>	<i>Allotment</i>
Q0	2	2
Q1	4	4
Q2	8	8

The default boost interval is 10. Budgets and boost interval must be positive whole numbers. Allotment may be smaller than quantum or not divisible by it. **Q used** counts this turn's CPU ticks; **A used** counts CPU ticks accumulated at this priority. Fractions show used/total.

A ready process has arrived and waits for the CPU. A future process has not arrived yet. Queues read from head to tail; [] means empty. Timeline cells each represent one tick. Examples are independent; assume no boost occurs unless stated.

2. Choosing a process

Rule 1. New arrivals join Q0

Append each new process to Q0's tail with both usage counters zero. Simultaneous arrivals enter in input order. Existing Q0 waiters keep their places.

Example. B waits in Q0. C and A arrive together, listed in that order.

```
Before arrivals:  Q0: [B]
New arrivals:      C, then A
After arrivals:   Q0: [B] [C] [A]
```

C and A each have Q used = 0 and A used = 0.

Rule 2. Choose the highest-priority ready queue

When the CPU is free, take the head of the highest-priority nonempty queue. Processes at the same priority take turns in Round Robin order. Service length does not change this choice.

Example. Q0 is empty; B heads Q1.

	Before dispatch	After dispatch
CPU	free	B
Q0	[]	[]
Q1	[B] [C]	[C]
Q2	[D]	[D]

C and D wait. D cannot run ahead of ready work in Q1.

Rule 3. Higher-priority work interrupts lower-priority work

At a boundary without a boost, higher-priority ready work preempts a lower-priority runner. Return the interrupted process to the **front** of its own queue and preserve both usage counters and remaining service. Same-priority arrivals do not interrupt a turn.

Example. A is running in Q1 when B arrives in Q0.

Before B arrives:	After B is dispatched:	
CPU: A in Q1	CPU: B in Q0	
Q0: []	Q0: []	
Q1: []	Q1: [A]	
A's counters:	Q used = 1/4	A used = 5/6
After preemption:	Q used = 1/4	A used = 5/6

When A resumes, it has 3 quantum ticks but only 1 allotment tick left, unless a boost resets its budgets.

If A were running in Q0 instead, B would wait in Q0 without interrupting A.

3. Accounting for CPU time**Rule 4. Charge only the process that runs**

For every execution tick, reduce remaining service by one and increase both usage counters by one. Waiting changes none of these values.

Example. A runs one tick with quantum 4 and allotment 6.

	Before	After
Remaining service	5	4
Q used	1/4	2/4
A used	1/6	2/6
Waiting process B:	all three values unchanged	

Rule 5. Completion takes precedence

At the next boundary, remove a process whose remaining service is zero. Completion wins over quantum expiry, allotment expiry, and boosting. A completed process is never requeued.

Example. Q0 quantum = 2, allotment = 2, boost interval = 2. A starts at 0 with service 2; B is waiting.

At t=2: A finishes; both budgets expire; boost is due.

```
1. Complete A      CPU: free    Q0: [B]
2. Boost waiting   CPU: free    Q0: [B]
3. Dispatch B      CPU: B       Q0: []
```

A is finished. It receives no new turn or budgets.

Rule 6. Exhausting allotment ends the priority level

At a boundary without a boost, append an unfinished process that has exhausted its allotment to the next lower queue. Reset both usage counters. In Q2, rejoin Q2's tail with fresh budgets. Preserve remaining service.

Allotment exhaustion takes precedence over quantum expiry and can end a turn partway through. If a boost is due, apply Rule 9 instead.

Example. A is alone, needs ten ticks, and starts in Q0 with quantum 2 and allotment 3. Q1 uses its default budgets of 4.

```
Q0 turn 1: 0 to 2 -> 2 CPU ticks used
Q0 turn 2: 2 to 3 -> 1 more tick; allotment is exhausted
```

Time	Result	Q used	A used	Service left
1	Continue in Q0	1/2	1/3	9
2	Start next Q0 turn	0/2	2/3	8
3	Move to Q1	0/4	0/4	7

At the lowest priority, an exhausted A would instead rejoin Q2:

```
CPU: A in Q2    Waiting Q2: [B]
               allotment expires
                   |
                   v
Before dispatch: Q2: [B] [A]
A stays in Q2; both usage counters reset to zero.
```

Rule 7. Quantum expiry alone ends only the turn

At a boundary without a boost, if quantum expires while allotment remains, append the unfinished process to the same queue's tail. Reset quantum usage; preserve allotment usage and remaining service.

Example. Q0 quantum = 2 and allotment = 6. A starts at 0 with service 5; B is waiting in Q0.

At $t=2$, before rotation:

CPU: A Q0: [B]

A: service left = 3 Q used = 2/2 A used = 2/6

After rotation, before dispatch:

CPU: free Q0: [B] [A]

A: service left = 3 Q used = 0/2 A used = 2/6

B runs next.

If C also arrives at 2: Q0 becomes [B] [C] [A]
before dispatch.

Rule 8. Keep the CPU busy when work is ready

Dispatch after a turn ends. A lone process may run again immediately, including after demotion. Idle only when both CPU and queues are empty. Future processes cannot run early. Stop when all processes finish.

Example. A arrives at 0, needs five ticks, and uses the default budgets.

```
Time 0   1   2   3   4   5
      +---+---+---+---+---+
CPU   | A | A | A | A | A |
      +---+---+---+---+---+
```

Time 0 to 2: A runs in Q0.

Time 2: A is demoted and immediately runs in Q1.

Time 5: A finishes. No idle tick was inserted.

If A instead arrived at 3, the CPU would idle until 3.

4. Priority boosts

Rule 9. Boost waiting and running processes

Boost at positive multiples of the boost interval, never at time 0. Complete any finished process first. Then:

1. Collect waiting queues in order Q0, Q1, Q2, preserving order within each.
2. Stop the unfinished running process and append it last, even if its quantum or allotment has just expired.
3. Place everyone collected in Q0 and reset both usage counters. Preserve remaining service.
4. Admit same-time arrivals, then dispatch normally.

Boosting overrides ordinary rotation and demotion at that boundary. A runner already in Q0 is treated the same way. With no other ready work or arrivals, a boosted runner is immediately selected again with fresh budgets.

Example 9a. A boost interrupts an ongoing turn

A is running in Q1 with budget remaining. B and C wait in Q1; D waits in Q2.

Before boost:	After boost, before dispatch:
CPU: A	CPU: free
Q0: []	Q0: [B] [C] [D] [A]
Q1: [B] [C]	Q1: []
Q2: [D]	Q2: []

All four: Q used = 0, A used = 0; service unchanged.

Dispatch: B runs; Q0 is [C] [D] [A].

If E arrives at the same boundary, before dispatch:

Q0: [B] [C] [D] [A] [E]

Example 9b. Allotment expiry coincides with a boost

A exhausts its Q0 allotment but still needs service. B, C, and D are waiting in Q0, Q1, and Q2 respectively.

```

Before handling the boundary:
CPU: A      Q0: [B]      Q1: [C]      Q2: [D]

Collect waiting work:  [B] [C] [D]
Append unfinished A:   [B] [C] [D] [A]
                        |       |
                        head    tail

Put all in Q0 with Q used = 0 and A used = 0.
Dispatch B; Q0 is [C] [D] [A]. Q1 and Q2 are empty.

```

A moves directly to Q0 without an intermediate demotion. Quantum expiry at a boost uses the same order. If A finishes instead, remove it first and boost only [B, C, D]. Any new arrival E joins after the boosted processes.

5. Events at the same boundary

First account for execution during the preceding tick.

Without a boost

1. Remove the running process if it has finished.
2. Check for allotment exhaustion, then quantum expiry.
3. Admit all arrivals at the current time, in input order.
4. Requeue any expired unfinished process, using Rule 6 or Rule 7.
5. If a process is still running, check for higher-priority preemption.
6. Dispatch if the CPU is free.

With a boost

1. Remove the running process if it has finished.
2. Collect waiting queues in order Q0, Q1, Q2.
3. Append the unfinished running process, even if a budget expired.
4. Put everyone collected in Q0; reset usage counters, preserve service.
5. Admit all arrivals at the current time, in input order.
6. Dispatch if any process is ready.

6. Worked example: separate quantum and allotment

A and B arrive at 0, in that input order, each needing ten service ticks. Use Q0 quantum 1 and allotment 4, default Q1/Q2 budgets, and boost interval 100. No boost occurs during this schedule.

Q0: four one-tick turns each

```
Time 0  1  2  3  4  5  6  7  8
      +---+---+---+---+---+---+---+---+
CPU   | A | B | A | B | A | B | A | B |
      +---+---+---+---+---+---+---+---+
```

Q1: one four-tick turn each

```
Time 8  9  10 11 12 13 14 15 16
      +---+---+---+---+---+---+---+---+
CPU   | A | A | A | A | B | B | B | B |
      +---+---+---+---+---+---+---+---+
```

Q2: two remaining service ticks each

```
Time 16 17 18 19 20
      +---+---+---+---+
CPU   | A | A | B | B |
      +---+---+---+---+
```

A's Q0 allotment usage after successive turns:

1/4 -> 2/4 -> 3/4 -> 4/4 -> move to Q1, reset to 0/4

Time Event

```
7       A enters Q1 with 6 service ticks left.
8       B enters Q1 with 6 left; A starts its Q1 turn.
12      A enters Q2 with 2 left; B starts its Q1 turn.
16      B enters Q2 with 2 left; A starts in Q2.
18      A finishes; B starts in Q2.
20      B finishes. Each process received 10 CPU ticks.
```